TEXTSAGE: Large Language Model (LLM) based tool for generating human-like responses to natural language inputs for networks not connected over the internet

SUBMITTED IN PARTIAL FULFILLMENT FOR THE REQUIREMENT OF THE AWARD OF DEGREE OF

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE

Submitted by

Turvash Singh (2200290120182)

Vinay Singh (2200290120192)

Chaitany Saini (2200290120060)

Supervised by

Dr. Anurag Mishra

Associate Professor

Session 2025-26

DEPARTMENT OF COMPUTER SCIENCE

KIET GROUP OF INSTITUTIONS, GHAZIABAD

(Affiliated to Dr. A. P. J. Abdul Kalam Technical University, Lucknow, U.P., India)

May 2026

DECLARATION

We hereby declare that this submission is our work and that, to the best of our knowledge and belief, it contains no material previously published or written by another person nor material which to a substantial extent has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgement has been made in the text.

Signature:

Name: Turvash Singh

Roll No.: 2200290120182

Signature:

Name: Vinay Singh

Roll No.: 2200290120192

Signature:

Name: Chaitany Saini

Roll No.: 2200290120060

Date: 28/04/2026

CERTIFICATE

This is to certify that Project Report entitled Offline Text Summarization Tool which is submitted by Turvash Singh, Vinay Singh, Chaitany Saini in partial fulfillment of the requirement for the award of degree B. Tech. in Department of Computer Science of Dr. A.P.J. Abdul Kalam Technical University, Lucknow is a record of the candidates own work carried out by them under my supervision. The matter embodied in this report is original and has not been submitted for the award of any other degree.

Date: 28/04/2026 **Supervisor**

Dr. Anurag Mishra
Associate Professor **(CS)**

Date: 28/04/2026 **Dr Ajay Kr Shrivastava**
(Dean CS)

ACKNOWLEDGEMENT

It gives us great pleasure to present the synopsis of the B.Tech Mini Project undertaken during B.Tech. Final Year. We owe a special debt of gratitude to Associate Professor Dr.Anurag Mishra, Department of Computer Science, KIET Group of Institutions, Delhi- NCR, Ghaziabad, for his constant support and guidance throughout our work. His sincerity, thoroughness, and perseverance have been a constant source of inspiration for us. It is only his/her cognizant efforts that our endeavors have seen the light of the day.

We also take the opportunity to acknowledge the contribution of Dr. Ajay Kumar Shrivastava, Dean of the Department of Computer Science, KIET Group of Institutions, Delhi- NCR, Ghaziabad, for his full support and assistance during the development of the project. We also do not like to miss the opportunity to acknowledge the contribution of all the department's faculty members for their kind assistance and cooperation during the development of our project.

Last but not least, we acknowledge our friends for their contribution to the completion of the project.

Date: 28/04/2026

Signature:

Name: Turvash Singh

Roll No.: 2200290120182

Signature:

Name: Vinay Singh

Roll No.: 2200290120192

Signature:

Name: Chaitany Saini

Roll No.: 2200290120060

ABSTRACT

Text summarization has emerged as a critical task in natural language processing, aiming to summarize large quantities of text into meaningful summaries. Within the context of this paper, fine-tuning and comparison for abstractive text summarization using the three most eminent transformer-based models—PEGASUS from Google, BART, and T5-medium—are considered. These transformer models were fine-tuned on pre-existing benchmark datasets to assess model performance in terms of fluent, coherent, and information-dense summarizations. The experiment procedure considered the reflection of model performance variation based on pre-training objective variations in transformer architecture. Evaluation for model performance took place using benchmark metrics like ROUGE-1, ROUGE-2, and ROUGEL scores. In addition to metric assessment, the experiment considered an assessment based on human evaluability in terms of generated summarizations. The outcome from the experiment illustrates the dominance of the PEGASUS model in terms of faithful-to-context, human-like summary capability. BART ranked second regarding efficiency in balance abstraction, in addition to factual cohesiveness. The T5-medium model, despite considering reduced capacity, exhibited remarkable efficiency in terms of reasonable accuracy. The experiment explores the feasibility aspect in terms of transformer architectural categorization for summarization capability.

TABLE OF CONTENTS

Section	
DECLARATION	ii
CERTIFICATE	iii
ACKNOWLEDGEMENTS	iv
ABSTRACT	v
LIST OF FIGURES	ix
LIST OF TABLES	x
LIST OF ABBREVIATIONS	xi
SDG Mapping	xii
CHAPTER 1 (INTRODUCTION)	1
1.1. Introduction to Project	1
1.2. Project Category	1
1.3 Objectives	1
1.4 Structure of Report	2
CHAPTER 2 (LITERATURE REVIEW)	3
2.1. Literature Review	3
2.2 Transformer-Based Models for Summarization	3
2.3 Problem Foundation	4
2.4 Research Gaps	4
CHAPTER 3 (PROPOSED METHODOLOGY)	5
3.1 System Overview	5
3.2 Unique Features of The System	7
3.3 Model Architecture Details	7
3.4 Hyperparameter Tuning	10
CHAPTER 4 (SYSTEM ARCHITECTURE)	12

4.1 Feasibility Study (Technical, Economical,Operational)	12
4.2 Software Requirement Specification	12
4.2.1 Data Requirement	12
4.2.2 Performance Requirement	13
4.2.3 Maintainability Requirement	13
4.2.4 Security Requirement	13
4.3 SDLC Model Used	13
4.4 System Design 4.4.1 Data Flow Diagram 4.4.2 Use Case Diagram	13
4.5 System Architecture	16
4.6 Design Approach	16
4.7 System Constraints	17
CHAPTER 5 (IMPLEMENTATION DETAILS)	18
5.1 Introduction Tools and Technologies Used.	18
5.2 Dataset Description	18
5.3 Implementation Challenges	18
CHAPTER 6 TESTING, AND MAINTENANCE	20
6.1 Testing Techniques	20
6.2 Test Case	20
CHAPTER 7 RESULTS AND DISCUSSIONS	22
7.1 Presentation of Results	22

7.2 Performance Evaluation	23
7.3 Discussion of Findings	23
7.4 System Limitations.	24
7.5 Comprative Analysis of Existing System	25
CHAPTER 8 CONCLUSION AND FUTURE SCOPE	36
8.1 Conclusion	27
8.2 Future Scope	27
8.3 Real World Application	28
REFERENCES	31
Research Paper Acceptance Proof	33
Reserch Paper	38
Project Report Plagiarism Report	49

LIST OF FIGURES

Figure No.	Description	Page No.
4.1	Data Flow Diagram DFD Level 0.	14
4.2	Data Flow Diagram DFD Level 1.	15
4.3	Use Case Diagram.	16

LIST OF TABLES

Table No.	Description	Page No.
6.1	Test Cases and Results of Offline Text Summarization Tool	20
7.1	Results on CNN/DailyMail Dataset	22
7.2	Results on XSum Dataset	22
7.3	Results on QMSum Dataset	23
7.4	Comparison of TextSage and Online Summarization Tools	26

LIST OF ABBREVIATIONS

Abbreviation	Full Form
NLP	Natural Language Processing
AI	Artificial Intelligence
ML	Machine Learning
GUI	Graphical User Interface
ROUGE	Recall-Oriented Understudy for Gisting Evaluation
CNN	Convolutional Neural Network
LSTM	Long Short-Term Memory
BART	Bidirectional and Auto-Regressive Transformers
T5	Text-to-Text Transfer Transformer
PEGASUS	Pre-training with Extracted Gap-sentences for Abstractive Summarization
PDF	Portable Document Format
TXT	Plain Text File

Sustainable Development Goal 9: Industry, Innovation, and Infrastructure

The Offline Text Summarization Tool (TextSage) aligns with SDG 9: Industry, Innovation, and Infrastructure, which emphasizes building resilient infrastructure, fostering innovation, and promoting inclusive and sustainable industrialization. The project contributes to this goal by developing an AI-based solution that functions without requiring continuous internet connectivity, thereby making advanced technology accessible in low-resource and remote environments.

By enabling offline processing, the system reduces dependency on cloud-based infrastructure and promotes efficient utilization of local computational resources. This supports the development of sustainable and decentralized technological solutions. Additionally, the use of modern Natural Language Processing (NLP) models reflects innovation in the field of artificial intelligence, encouraging the adoption of intelligent systems in real-world applications such as education, research, and business.

CHAPTER 1

INTRODUCTION

1.1 Introduction to Project

As digital information expands at an alarming rate, the skill of deriving valuable information out of large amounts of text has gained significance. Text summarization is a Natural Language Processing (NLP) method to produce concise and coherent summaries and retain the necessary information of the original text automatically. Old forms of summarization were based on extractive techniques, the method that picked out significant sentences of the original text. Nevertheless, more recent developments in deep learning and Transformer-based networks have made abstractive summarization, which models can create human-like summaries by learning the semantic meaning of the text, possible. All the available summarization tools are cloud based and need internet connectivity, which brings up the issues of privacy, latency and availability. The project aims to create an offline text summarization application, which allows users to obtain summaries on their devices with fine-tuned Transformer models.

1.2 Project Category

TextSage the Offline Summarization Application is a desktop-based application written in Python and Tkinter to enable a user to manually input the text or upload a text file to be summarized. A collection of models including PEGASUS, BART, and T5 fine-tuned versions is provided by the system so that individuals have the power to examine the outcomes of various structures.

All vanilla models are trained on three standard benchmark datasets:

• CNN/DailyMail

• XSum

• QMSum

1.3 Objectives

The primary goals of the project are:

1. The development and creation of an offline text summarizer system.
2. To train several models of Transformers to be optimized to perform summarization.
3. To add a handy desktop interface to type in and create a summary.
4. To gauge the model performance in terms of ROUGE. To compare PEGASUS, BART, and T5 performance

in various datasets.

1.4 Structure of Report

The following report is structured into the following:

Chapter 1: Introduction— It introduces TextSage, problem context, project objectives, and report structure.

Chapter 2: Literature Review—Reviews literature on mining safety and regulatory technologies, solutions available, and gaps in the research and presents the problem statement.

Chapter 3: Proposed System—Proposed System- Discusses the proposed system, system workflow, system dataset preparation, AI/NLP methodology, processing pipeline, and system key features.

Chapter 4: Requirement Analysis and System Specification—Requirement Analysis and System Specification- Discusses feasibility, Software Requirements Specification (SRS), SDLC model and high-level system and database design.

Chapter 5: Implementation—Provides a description of the tools and technologies employed and implementation approach of the dataset.

Chapter 6: Testing and Maintenance—Summarizes testing methods, major test cases.

Chapter 7: Results and Discussions—Details results of performance, evaluation and major findings.

Chapter 8: Conclusion and Future Scopes—Gives the conclusion and future improvements.

CHAPTER 2

LITERATURE REVIEW

2.1 Literature Review

Problems of consistent facts, reliability of evaluation and diversity of data continue to be a dynamic research issue, as recent surveys and evaluation research suggest. The given project helps address these gaps by optimizing different Transformer-based models with diverse benchmark datasets and embedding them into an offline desktop application that allows them to compare models and make local inferences.

Several Studies have stated that summarization methods moved forward from old mathematical ways of counting to deep learning and Transformer-based systems [13], [9]. It has been observed that summarization methods gained better flow of words and logic because the understanding of the surrounding meaning increased [13], [9].
These articles also highlight the significance of benchmark datasets and evaluation measures in facilitating the research on summarization.

Data sets have been instrumental in the process of training and testing summarization models. To aid the study in machine reading as well as summarization tasks, big volume reading-understanding datasets like CNN/DailyMail were launched [4]. Even more specialized datasets like QMSum have extended the study of meeting and long-document summarization further [14]. Additional datasets like Newsroom have given massive lists of summaries with divergent extractive methods [17].

Summarization systems are often evaluated based on ROUGE metrics, measures of overlap between generated and reference summaries, and are also one of the most popular metrics used to benchmark the performance of summarization systems [23].

2.2 Transformer-Based Models for Summarization

The beginnings of the Transformer architecture would be a breakthrough in sequence modeling. The self-attention mechanism has been suggested by Vaswani et al., which allows models to efficiently capture long-range dependencies and enhances their performance in a wide variety of NLP tasks [3].

Based on the Transformer architecture, a number of sequence-to-sequence models that are pre-trained have set state-of-the-art results in summarization. BART also added a denoising pretraining task that enhanced natural language generation tasks, such as summarization [6]. T5 was able to generalize a wide range of NLP tasks to a

text-to-text model and showed good transfer learning properties and multi-task scaling [7]. PEGASUS proposed a special type of pretraining algorithm that allows models to be summarized by masking and predicting key sentences, showing a large performance increase in abstractive summarization tasks [8].

Hierarchical Transformers and specialized architectures to process long documents and multi-document summarization have been investigated in other studies [16], [22]. To enhance the processing of long textual sequences, long-input summarization methods, including dynamic latent extraction suggested [15].

2.3 Problem Foundation

Previous neural summarization methods incorporated sequence-to-sequence models that added attention mechanisms and pointer-generator networks which enhanced the processing of rare or out-of-vocabulary words and repetition reduction [1]. They also used reinforcement learning methods to maximize evaluation measures and coherence in summaries [2].

Although these developments are made, there are still a number of challenges. A significant weakness is the problem of factual consistency. Experiments have demonstrated that, abstractive summarization models can produce fluent and factually incorrect information commonly known as hallucinations [10], [11]. More recent has suggested assessment schemes to evaluate the domain-specific summarization tasks in terms of factual accuracy [12].

Other drawbacks of most current summarization systems are that they are based on cloud-based infrastructure, which raises some issues regarding privacy, latency, and accessibility. This has driven the development of effective and implementable summarization solutions.

2.4 Research Gaps

Despite the important advancements that have been achieved in abstractive summarization, there are still a number of gaps in practical and comparative research.

To start with, most of the studies are aimed at enhancing a single model as opposed to comparing several state-of-the-art models on a wide range of datasets within a single framework. Relative comparisons of models including BART, T5, and PEGASUS in similar fine-tuning and evaluation conditions are still relatively scarce.

Problems of consistent facts, reliability of evaluation and diversity of data continue to be a dynamic research issue, as recent surveys and evaluation research suggest[9][13]. The given project helps address these gaps by optimizing different Transformer-based models with diverse benchmark datasets and embedding them into an offline desktop application that allows them to compare models and make local inferences.

CHAPTER 3

PROPOSED METHODOLOGY

3.1 System Overview

The proposed system is an offline text summarization program, providing users the ability to generate summaries offline using fine-tuned Transformer-based models. It integrates some elements of summarization models, including PEGASUS, BART, and T5, and the user can select a model and generate summaries through a graphical user interface (GUI) on the desktop. After model training and configuration, the system becomes completely offline. It enables the user to type in the text or load the text files and the input text is processed by the application to provide a summary and is shown on the interface. The quality of generated summaries is measured by ROUGE metrics that measure the performance of the models.

3.1.1 Dataset Description

Various benchmark datasets that are widely used in summarization research were optimized with the models to ensure. powerful training and assessment.

CNN/DailyMail Dataset: It consists of a set of news articles that have human summaries and can be used as a reference point and training data to train abstractive summarization models. It contains long-form news content and multi-sentence summaries, and is suitable in general-purpose summarization tasks.

XSum Dataset: The XSum data is news stories with very abstractive, single-sentence summaries. This dataset proves is more difficult for models, as the models have to produce summaries of the text that will fit the overall idea of the text.

QMSum Dataset: QMSum is prepared to be useful for query-based and long-document summarization, specially when dealing with meeting transcripts and multi-domain discussion.

The dataset may be utilized to evaluate the models in terms of their functionality with more complicated and lengthy input format. The use of multiple datasets contributes to better generalisation of the model and allows to compare the performance of the model in different settings of summarisation. The multiple datasets are useful to enhance the generalization of the model and enable the performance to be compared in various summarization scenarios.

3.1.2 AI Model Development and Fine-Tuning

In this project, three Transformer-based sequence-to-sequence models were picked:

- PEGASUS
- BART
- T5

These models were selected due to their good performance on abstractive summarization tasks and the presence of pre-trained weights.

The steps of the fine-tuning process were as follows:

1. Loading pre-trained tokenizers and models.
2. Preprocessing datasets: tokenizing text and summaries.
3. Specifying training parameters, including batch size, learning rate, and epochs.
4. Supervised learning models of summarization datasets with labels.
5. save offline inference of the fine-tuned models.

Fine-tuning allows the models to fit domain-specific patterns and enhances the accuracy of summarization of the chosen datasets.

3.1.3 Feature Implementation

The features of the application are as follows:

PEGSUS vs. BART vs. T5.

- Input text summarization
- Support for TXT file input Offline working without internet connectivity.
- Visuals of generated summaries in the GUI.

The reason behind the introduction of these features was to offer flexibility and usability to end users and still ensure efficient model inference.

3.1.4 Data Processing Pipeline

The pipeline that summarizes is composed of the following steps:

1.Input Acquisition:

The text that is fed into the system is by manual input or a text file.

2.Preprocessing:

The tokenizer of the chosen model cleans up and tokenizes the input text.

3.Model Inference:

The tokenized input is fed to the fine-tuned model to produce a summary.

4.Post-processing:

The tokens created are decoded to readable text.

5.Output Display:

The last summary is presented in the application interface.

This pipeline guarantees a smooth and uninterrupted processing of the input.

3.1.5 System Development and Integration

Python was the major programming language that was used to develop the system. During the loading, training and inference of models, Hugging Face Transformers library was utilized, and the deep learning backend was PyTorch.

The Tkinter was used to implement the graphical user interface, and it enables users to interact with the system with ease. The trained models were incorporated in the application in a way that the user can choose a model and produce summaries without further configuration.

3.2 Unique Features of The System

TextSage has the following features that make it differentiate itself:

1. Fully Offline System: The summarization tool works without any internet connectivity on the user local machine.
2. Privacy: The user data is never transmitted over the internet and user don't have to signup to download the application.
3. Different Models: TextSage offers multiple state-of-the-art fine-tuned models and user can select any of them to summarize the text.
4. Lightweight Graphical User Interface: TextSage GUI is built with Tkinter which offers simple and intuitive interface.
5. Local File Handling Support: The system allows user to upload text file and after inference the user can download the summarize content in form of text file format.

3.3 Model Architecture Details

Transformer-based systems have made a huge leap in the Natural Language Processing (NLP) field as they allow models to comprehend and generate human-like text effectively. In contrast to the traditional step-by-step models like Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks,

Transformers operate on entire sequences at the same time. This capability of parallel processing enables them to capture long-range dependencies better and greatly minimizes training time.

The main strength of Transformer models is that they can model contextual relationship of words irrespective of their location in a sentence. This renders them especially well-suited to activities like text summarization, where it is important to understand the context as a whole.

3.3.1 Transformer Architecture Overview

Transformer architecture is made up of two basic components; the Encoder and the Decoder.

The Encoder will handle the input text and transform it into a sequence of contextual embeddings. These embeddings are representations of the semantic meaning of each word within the whole input sequence. The encoder consists of several identical layers, each with a multi-head self-attention mechanism with a feed-forward neural network.

The Decoder, in its turn, produces the output sequence (the summary, in this instance) according to the input that was encrypted. It is also multi-layered with masked self-attention and encoder-decoder attention mechanisms added to the layers so that the output generated by the multi-layered model can be contextually significant and coherent.

The encoder-decoder interaction enables the model to produce semantically meaningful summaries.

3.3.2 Self-Attention Mechanism

The most important part of the Transformer architecture is the self-attention mechanism. It enables the model to analyze the significance of each word in a sentence compared to all other words.
There are three vectors that self-attention works with:
Query (Q): The word in progress.
Key (K): Reflects all the other words in the sequence.
Value (V): The actual information of the words is found in it.

The model calculates the attention scores by matching the Query vector and each of the Key vectors. These scores are used to decide the extent to which a given word should be paid attention to when creating the representation of a given word. The last is a weighted average of the Value vectors.

This process allows the model to reflect intricate associations like subject-object associations and contextual meanings, even in very long sentences.

3.3.3 Positional Encoding

Transformers do not process the input data in order; therefore, they do not have prior knowledge of the order of words. Positional encoding is added to the input embeddings to overcome this limitation.

Positional encoding encodes the position of the position in the sequence with a unique representation based on sine functions or learned embeddings. This enables the model to distinguish the words with regards to their location and preserve the sentence structure.

An example is that the interpretation of the sentence the cat chased the mouse differs with the meaning of the sentence the mouse chased the cat and positional encoding assists the model in capturing this difference.

3.3.4 PEGASUS Model

PEGASUS is a Transformer model that is specially created to summarize abstractive text. It presents a new pre-training task called Gap Sentence Generation (GSG).

In this method, a document is taken and important sentences are deleted and then the model is trained to guess the deleted sentences using the rest of the text. This is very similar to the summarization task and PEGASUS is very effective in producing high quality summaries.

This model has high productivity in generating concise, coherent and contextually correct summaries. It is especially appropriate with long documents as its architecture enables it to absorb local and global contextual information.

3.3.5 BART Model

BART (Bidirectional and Auto-Regressive Transformers) is a denoising autoencoder that unites the advantages of bidirectional encoders (such as BERT) and autoregressive decoders (such as GPT).

In training, corruptions of the input text are introduced intentionally with a number of noise functions, including token masking, sentence shuffling, and deletion. The model is further trained to generate the original text given the corrupted one.

This method of training will allow BART to build up a strong language structure and context knowledge. Consequently, it is effective in text generation tasks, such as summarization, translation, and question answering.

BART works best with noisy or missing input data and is thus a flexible model that can be applied to real-world scenarios.

3.3.6 T5 Model

The T5 (Text-to-Text Transfer Transformer) model presents a single model of processing various NLP tasks by transforming them to a text-to-text format.

In this method, input and output are considered as texts. To illustrate, one of the summarization tasks would be phrased as follows:

Summarize: [input text] → [summary].

This combined method makes the training process easier and enables the model to generalize on various tasks. T5 is scalable and flexible and exhibits good performance in many NLP applications.

Though it might not be as good as dedicated models such as PEGASUS in summarization tasks, it offers a balanced performance in various fields.

3.4 Hyperparameter Tuning

Hyper-parameter tuning is a very important process in ensuring machine learning models are optimized. Hyper-parameters are configuration parameters that are predetermined prior to the onset of the training process and greatly determine the manner in which the model would learn the data.
A number of hyper-parameters in this project were selected and optimized to provide the best performance of the summarization.

3.4.1 Learning Rate

Learning rate defines how big the steps are in which the model changes its weights during the training process. A very large learning rate may lead to the model converging too soon to an in-optimal solution, and a very small learning rate may lead to a slower convergence, and longer training time. To have stable and effective training, it is necessary to choose the right learning rate.

3.4.2 Batch Size

The number of samples trained per iteration is referred to as batch size. Smaller batch sizes enable the model to generalize more and can be more expensive to train. Smaller batch sizes are less efficient to compute, but can provide more stable learning.

3.4.3 Number of Epochs

The number of epochs determines the number of times the model is run on the full dataset in training. The number of epochs should be optimal to avoid the model over-fitting to the training data.

3.4.4 Maximum Token Length

The maximum number of tokens that the model can handle in one input sequence is determined by its token

length. Long sequences have more contextual information but consume more computational resources.

3.4.5 Decoding Strategies

1.Inference involves different decoding strategies to make summaries:

2.Greedy Decoding picks the most likely token in each step.

3.Beam Search considers a variety of potential sequences and picks the most suitable overall output.

4.Beam search tends to come up with more coherent and better-quality summaries than greedy decoding.

3.4.6 Impact on Performance

Effective hyper-parameter optimization results in better model performance in the form of:

1. Correctness of the summaries generated.

2. Fluency and readability.

CHAPTER 4

REQUIREMENT ANALYSIS AND SYSTEM SPECIFICATION

4.1 Feasibility Study (Technical, Economical, Operational)

The feasibility study assesses the applicability of the TextSage system from technical, economical, and operational aspects to assure effective adoption in the mining industry.

4.1.1 Technical feasibility.

The system is built using Python, Tkinter, Pytorch, and NLP-based Large Language models. It works on standard desktops and mobile devices without internet connectivity and does not require any extra hardware. The modular design enables for simple future modifications, such as adding new fine-tuned and multi-lingual models.

4.1.2 Economic feasibility.

TextSage inexpensive since it employs open-source LLM's. It reduces incident response times, manual consultation costs, and document usage.

4.1.3 Operational feasibility.

The system employs an easy-to-use structure which helps in using the system with minimal technological knowledge. The tool gives help around the clock, and fine-tuned models assure the factual information.

4.2 Software Requirement Specification.

The SRS describes the TextSage system's functional and non-functional requirements. It is used as a guide for system design, development, and testing.

4.2.1 Data Requirement

1. Input Data: The System can handle data in 2 formats.

Plain text data

Text File

2. Training Data: The fine-tuning of the vanilla models is done of standard summarization dataset.

CNN/DailyMail, XSUM and QMSUM.

4.2.2 Performance requirements.

1. The System must summarize the content in 7-8 seconds.
2. The System can handle the very short given text for summary.
3. Different models may take different inference time depending on the size of the input.

4.2.3 Maintainability Requirement

It must be expandable and flexible so that it can be easily modified and maintained. New legislation and features should be implemented without affecting the old components. It should be properly documented and clean code practices ought to be maintained so that future updates can be easily made.

4.2.4 Security requirements

1. Although the TextSage does not need internet connection to work, but certain security measure can be taken to prevent any miss happening.
2. The system is intended to be for single user local access and does not require authentication system.
3. User input data is not stored parmanently unless user wants to.
4. The System can handle files in text format.
5. Input file is validated before processing to prevent any harm.
6. User can save the generated summary in form of text file.
7. Integrity is applied while training of models.
8. System must handle large text inputs and large text files effectively to prevent any excessive usages of CUP or GPU.
9. The system offers check for GPU in users hardware for faster inference if GPU is present then the system will use it if not then system will use CPU.
10. Input size limits must be enforced to prevent unwanted crash of application.

4.3 SDLC Model Used

TextSage is developed with the help of the Agile SDLC model. Agile enables the incremental development, constant testing, and continuous feedback of the stakeholders. The project will have stages which include requirement analysis, design, development, testing and deployment. This will be flexible, enhancements will be quicker, and adaptability to regulatory changes will be enhanced.

4.4 System Design

System design establishes the architecture and design of TextSage. The System made upon modular, standalone Architecture where GUI is built with the help of Tkinter and it directly interacts with application logic. The System process the text data provided by the user and pass it to the text summarization engine which is

implemented using Pytorch and Huggingface Transformers. The system do all the process locally on the user machine without relying of any external API's which guarantees the privacy.

4.4.1 Data Flow Diagram

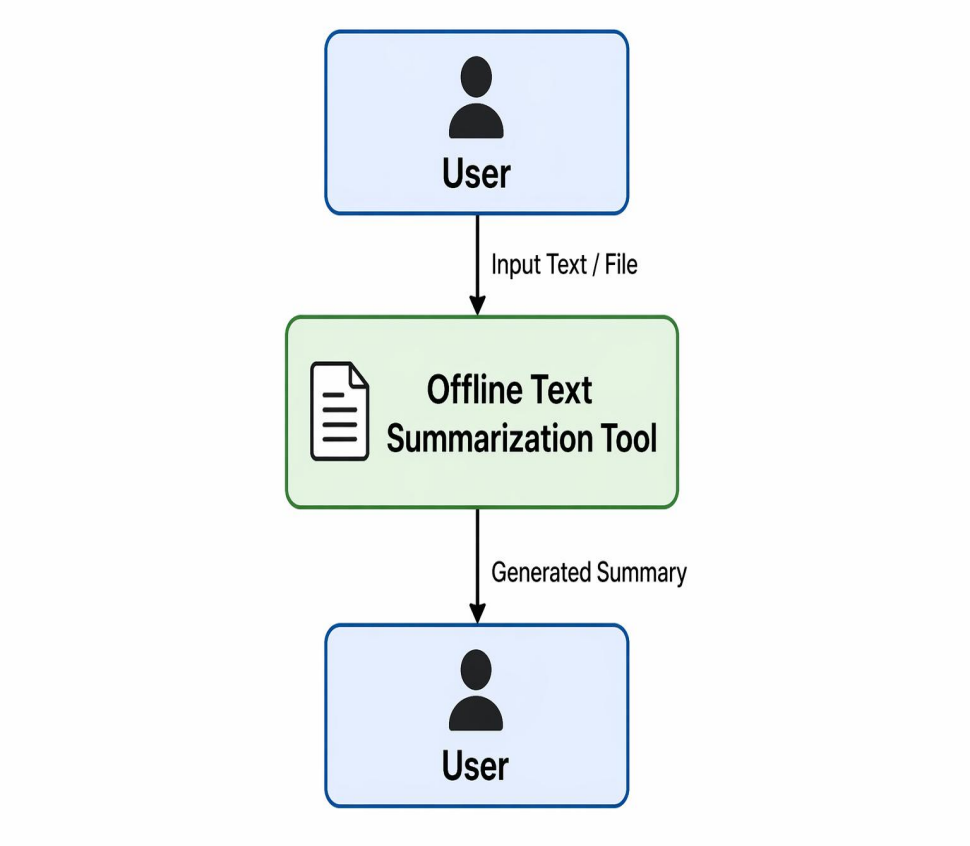

Fig 4.1: DFD Level 0 (Context Diagram)

"This diagram represents the high-level overview of the Offline Text Summarization Tool, illustrating the interaction between the user and the system. The user provides input text or files, and the system processes the data to generate and return a summarized output."

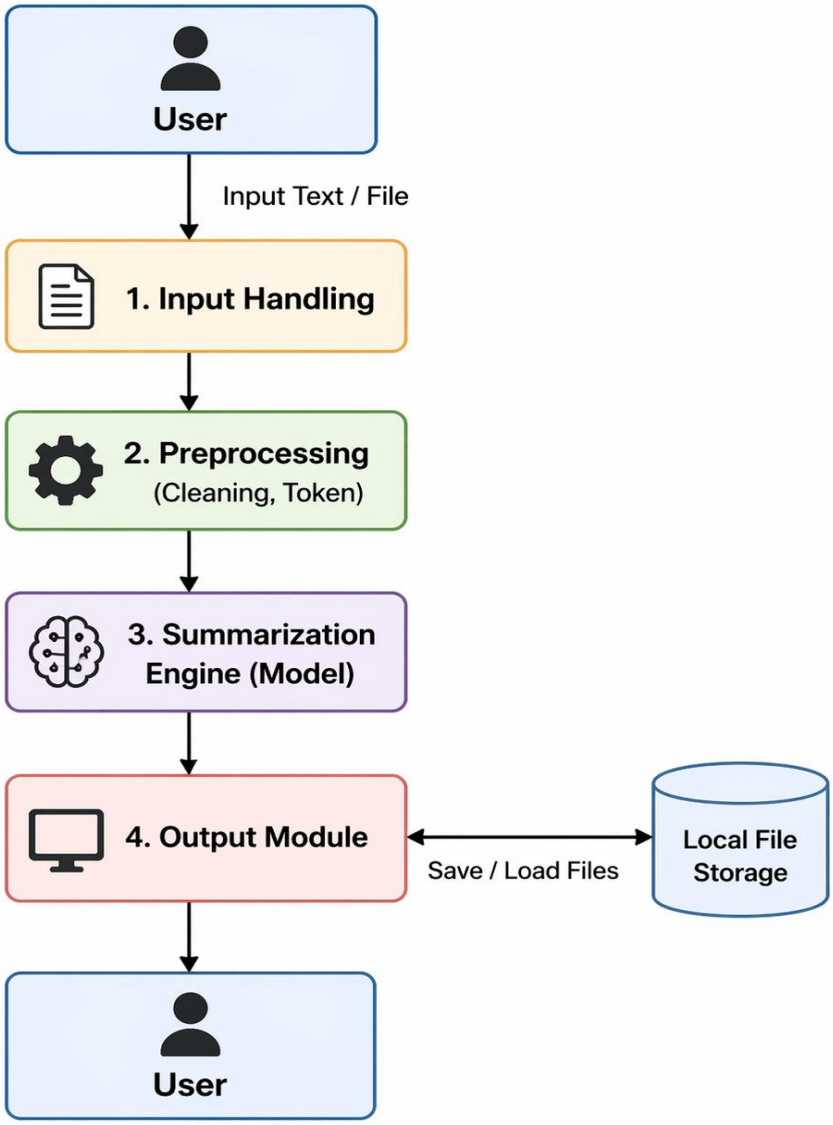

Fig 4.2: DFD Level 1

"This diagram provides a detailed breakdown of the internal workflow of the Offline Text Summarization Tool, including input handling, text preprocessing, summarization using the model, and output generation. It also depicts interaction with local file storage for saving and retrieving data."

4.4.2 Use Case Diagram

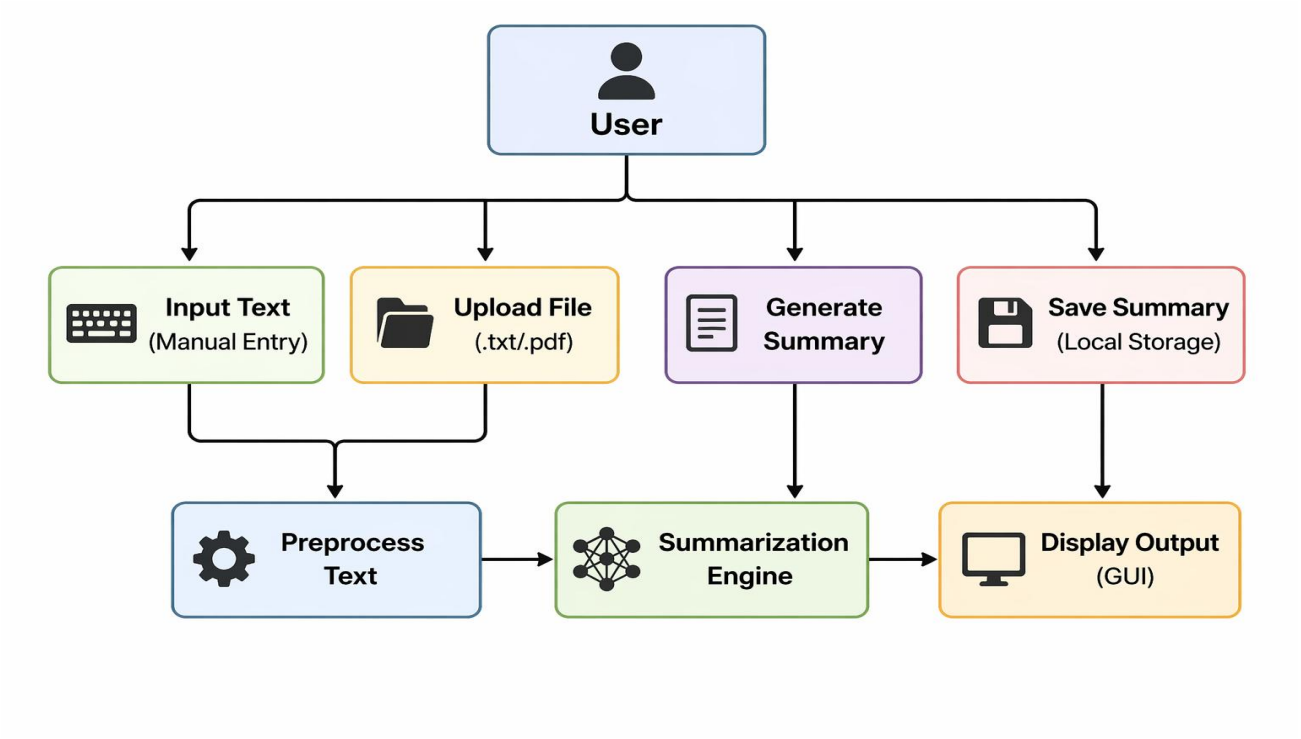

Fig 4.3 Use Case Diagram

"This diagram illustrates the functional interactions between the user and the system, including actions such as entering text manually, uploading files, generating summaries, and saving the summarized output to local storage."

4.5 System Architecture Explanation

The TextSage system is developed based on a modular architecture which guarantees scalability, maintainability and easy integration. All elements of the system have a certain role, which is part of the workflow.

4.6 Design Approach

1. Modular Design: Each Component is very loosely coupled with each other.
2. Offline-first Architecture: No module of the system is dependant on the internet connectivity for any kind of processing.
3. Scalability: Because of the loosely coupled nature of the system the models can be replaced with sate-of-the-art models without affecting any other component of the system.

4.6.1 Workflow Description

1. The system has a systematic workflow:
2. The graphical user interface lets the user manually type in the text or upload a text file.
3. The input text is fed to the preprocessing stage where it is cleansed and tokenized.
4. The processed input is sent to the chosen Transformer model.

5. The model produces a summary according to the patterns learn.

6. The resultant summary is presented to the user.

4.6.2 Module Description

1. Input Module

In charge of taking in user input in different formats like text files and plain text.

2. Preprocessing Module

Does text cleaning, normalization, and tokenization to preprocess the data to be used in the models.

3. Model Inference Engine

Runs the trained models (PEGASUS, BART, T5) and creates summaries.

4. Output Module

Shows the summary that has been generated in the graphic interface and enables users to save the output.

4.7 System Constraints

1. Performance of the summrization tool depends upon the hardware of the user(CPU/GPU, RAM).
2. If GPU is not present in the users hardware then inference time may increased.
3. Large input text may affect the processing time.
4. Model size may impact storage and loding time.

CHAPTER 5

IMPLEMENTATION DETAILS

5.1 Introduction to Tools and Technologies Used

TextSage was implemented with the use of the latest ML development technologies to guarantee the scalability, performance, and maintainability. Tkinter as used to create the frontend of the system and offer a responsive and interactive user interface to the system, allowing users to interact with the LLM's, allows user to select model of their own choice and summarize the text. Text-Summrization Engine is built with the help of Pytorch and HuggingFace Transformers. The System is constrained to produce summary between 30-180 words and if the user text is less than 30 words the system will print the oneline summary.

5.2 Dataset Description

CNN/DailyMail Dataset: It consists of a set of news articles that have human summaries and can be used as a reference point and training data to train abstractive summarization models. It contains long-form news content and multi-sentence summaries, and is suitable in general-purpose summarization tasks.

XSum Dataset: The XSum data is news stories with very abstractive, single-sentence summaries. This dataset proves is more difficult for models, as the models have to produce summaries of the text that will fit the overall idea of the text.

QMSum Dataset: QMSum is prepared to be useful for query-based and long-document summarization, specially when dealing with meeting transcripts and multi-domain discussion.

The dataset may be utilized to evaluate the models in terms of their functionality with more complicated and lengthy input format. The use of multiple datasets contributes to better generalisation of the model and allows to compare the performance of the model in different settings of summarisation. The multiple datasets are useful to enhance the generalization of the model and enable the performance to be compared in various summarization scenarios.

5.3 Implementation Challenges

The creation of the TextSage system was associated with several technical and practical issues, which were mainly connected with the complexity of computations of Transformer-based models and limitations of offline implementation. Conquering these issues was vital in order to have an efficient, useful and working system.

5.3.1 Large Model Size

PEGASUS, BART, and T5 are transformer-based models and are typically big in size, potentially taking several gigabytes of memory. The problem was that it was difficult to manage these models locally due to storage space and incompatibility of the systems.

Moreover, the loading of these models into memory had to be optimised carefully since lack of memory would cause system crashes or even slow performance.

5.3.2 Hardware Limitations

Because the system will be intended to run offline, it will have to be run efficiently on normal computing hardware without utilizing cloud-based GPU resources. Transformer models are however computationally demanding and work better in high-end GPUs.

In response to this, optimizations like smaller batch size and efficient memory management were introduced to make sure that it could smoothly run on CPU-based systems.

5.3.3 Slow Inference Time

Creation of summaries on large input texts can be time-intensive, particularly on systems that are not equipped with GPU acceleration capabilities. The longer the input and the more the model complexity, the larger the inference time.

To overcome this, methods like multi-threading and efficient decoding methods were employed to decrease the processing time and enhance responsiveness.

5.3.4 Data Preprocessing Complexity

Training datasets involved a lot of preprocessing such as cleaning, tokenization and formatting. The management of inconsistencies, missing values, and formatting were upheld when handling large datasets like CNN/DailyMail and QMSum.

5.3.5 Token Limit Constraints

Transformer models are limited by a fixed maximum number of tokens, which limits the amount of input text that can be handled simultaneously. This was a limitation and this was problematic with long documents.

To address this, long texts would be broken into small bits prior to processing and the results would be merged to produce an end result which was a summary.

CHAPTER 6

TESTING AND MAINTENANCE

6.1 Testing Techniques

The system was put through numerous tests to make sure that it is right and user friendly:

a) Checking summrization tool responses was done through predefined legal questions and checking whether the responses were correct, topical and contextual.

b) The incident reporting feature was tested simulating the real-world incident scenarios to check adequate report creation, alerts and control by role.

c) User feedback from students and professionals was then used to iteratively improve the system's usability and overall performance.

6.2 Test Cases and testing.

A detailed testing plan was carried out to improve the reliability and robustness of the TextSage system. A series of test cases was used to test the system, including various input cases and edge cases.

Table 6.1: Test Cases and Results of Offline Text Summarization Tool.

• Test Case	• Input Type	• Expected Output	• Result
• TC1	• Short Text	• Concise and accurate summary	• Passed
• TC2	• Long Article	• Detailed and coherent summary	• Passed
• TC3	• File Input	• Proper file processing and summary	• Passed
• TC4	• Empty Input	• Appropriate error handling	• Passed
• TC5	• Special Characters	• Clean and readable output	• Passed

"This table summarizes test cases used to evaluate the Offline Text Summarization Tool across different input scenarios. It highlights expected and actual results, demonstrating the system's accuracy and robustness."

6.2.1 Functional Testing

The functional testing was performed to ensure every aspect of the system has been functioning in its respective capacity. These encompassed input handling testing, model execution and generation of output.

6.2.2 Performance Testing

The performance testing was used to test the efficiency and response time of the system when the size of the input changed. It made sure that the system can sustain acceptable performance levels even with large inputs.

6.2.3 User Acceptance Testing

The usability and effectiveness of the system were evaluated based on the feedback of the users. This aided in determining areas of improvement and also made sure that the system is able to meet the expectations of users.

CHAPTER 7

RESULTS AND DISCUSSION

7.1 Presentation of Results

To evaluate the quality of the generated summaries, the models were assessed using the ROUGE (Recall-Oriented Understudy for Gisting Evaluation) metric. ROUGE is a widely used evaluation measure for summarization tasks and compares the overlap between generated summaries and reference summaries.

The following metrics were used:

ROUGE-1: Measures overlap of unigrams (individual words).

ROUGE-2: Measures overlap of bigrams (pairs of words).

ROUGE-L: Measures the longest common subsequence, reflecting fluency and structural similarity.

The performance of PEGASUS, BART, and T5 models was evaluated on three benchmark datasets: CNN/DailyMail, XSum, and QMSum.

Table 7.1 Results on CNN/DailyMail Dataset

Model	ROUGE-1	ROUGE-2	ROUGE-L
PEGASUS-large	44.5	21.3	41.2
BART-large	43.2	20.4	40.1
T5-Medium	40.1	18.1	37.3

"This table compares the performance of different summarization models using ROUGE metrics, highlighting the effectiveness of each model."

Table 7.2 Results on XSum Dataset

Model	ROUGE-1	ROUGE-2	ROUGE-L
PEGASUS	47.8	24.5	39.8
BART	45.1	22.2	36.7
T5-Medium	41.5	19.0	32.8

"This table presents the ROUGE scores of different summarization models, showing their comparative performance in generating accurate summaries."

Table 7.3 Results on QMSum Dataset

Model	ROUGE-1	ROUGE-2	ROUGE-L
PEGASUS	39.2	12.5	33.0
BART	41.8	14.3	35.7
T5-Medium	35.4	10.2	29.5

"This table shows the ROUGE scores of different models on the QMSum dataset, comparing their effectiveness in summarizing meeting-based content."

These results indicate that all models performed effectively, with PEGASUS and BART generally achieving higher scores compared to T5.

7.2 Performance Evaluation

Various metrics are used to measure the performance of Summrization tool such as ROUGE (Recall-Oriented Understudy for Gisting Evaluation) to measure the quality if generated summary. The standard dataset are used to train and test the model performance. CNN/DailyMail and XSUM dataset are used to train the model to generate abstractive summary and forces the models to remember the longer context window and QMSUM dataset is used to train the model of for meeting transcripts after the training the ROUGE scores of each model is increased by 3-4. The summarization tool creates different thread to generate summary which reduce the inference time by 18% and it also prevents the UI from freezing while inferencing.

7.3 Discussion of Findings

As the experimental results indicate, Transformer-based models are effective in the abstractive summarization task on various datasets.PEGASUS has the largest ROUGE scores on CNN/DailyMail and XSum, which corresponds to its pretraining aim, which is explicitly aimed at summarization. BART showed competitive results and generated consistent summaries with datasets. T5 had relatively lower ROUGE scores but with consistent performance and high generalization capability.

The differences in performance between datasets suggest that the characteristics of datasets are very important determinants of model behaviors. To illustrate, XSum demands very succinct summaries, which prefer PEGASUS, whereas QMSum has longer and more complicated inputs, resulting in comparatively lower scores of all models.

The other key finding is that fine-tuned models used in offline inference offer a high-quality summarization without the need to be connected to the internet. This shows the possibility to use Transformer-based summarization systems in local settings.

Generally, the findings validate that the suggested system can produce quality summaries and provide flexibility, privacy, and usability in its offline desktop interface.

7.4 System Limitations.

Although the TextSage system has excellent results in terms of text summarization, it has some weaknesses that must be considered.

7.4.1 Generated Summaries Hallucination.

Transformer based models have a major weakness in the form of hallucination, i.e., factually incorrect or misleading information.These Hallucination can be minimized by introducing the human-feedback-in-the-loop but can't be prevent fully.The system offers only have limited number of models, and it maybe possible that these models can't perform as expected.User have no way to check weather the generated summary is correct or not other than reading the generated summary.

7.4.2 Language Limitation

The existing system is mainly English text based. It does not facilitate multi-lingual summarization and this restricts its use in a multi-lingual setting. The system cannot understand and other language than English.

7.4.3 High Computational Requirements

1. Even with optimizations, the system continues to be very demanding on computational resources to operate effectively, particularly handling large documents.

2. The system needs expensive hardware to run smoothly on it.

3. Even though the system can run of CPU but the inference time may going to increase for CPU.

4. The system can run perfectly fine with inference time less than 10 sec on hardware with 4GB VRAM.

7.4.4 Absence of Real-time Learning.

1. The system lacks automatic updating and improvement with the new data. Any updates need to be retrained by hand.

2. Real time learning makes model more accurate and dynamic with the user input.

3. The system could learn the behaviour of user and accordingly provide result with real time learning.

4. The performance is limited to the knowledge captured during initial training or fine-tuning on datasets like CNN/DailyMail or XSum.

5. The system does not incorporate user corrections or preferences to improve future summaries.

6. Any improvement requires explicit retraining or fine-tuning, which is time-consuming and resource-intensive.

7. The system cannot tailor summaries based on individual user preferences or usage patterns.

8. Unlike cloud-based AI systems, the model does not evolve or improve collectively from multiple users' data.

7.4.5 Reliance on Training Data.

Training datasets are crucial in the quality of generated summaries. The data set has possible biases or constraints that can influence the quality of output.Any biases present in datasets (e.g., CNN/DailyMail, XSum) may be reflected in the generated summaries, affecting neutrality and fairness. Models trained on general datasets may not perform well on specialized domains such as legal, medical, or technical documents. Since the model is trained on static datasets, it may lack awareness of recent trends, terminology, or newly emerging topics. If certain words, phrases, or contexts are underrepresented in the training data, the model may generate incomplete or less accurate summaries. The system does not automatically update its knowledge base with new data unless explicitly retrained.

7.5 Comparative Analysis to Existing Systems.

A comparative study was done to compare the performance and characteristics of the TextSage system with the available online summarization systems.The Offline Text Summarization Tool (TextSage) was evaluated against existing online summarization systems to understand its strengths and limitations. Unlike most online tools that require a constant internet connection, TextSage operates entirely offline, making it more reliable in restricted or low-connectivity environments. It also offers enhanced data privacy, as user data is processed locally rather than being transmitted to external servers. In terms of cost, TextSage is free to use, whereas many online platforms follow a subscription-based model. However, online tools may have an advantage in processing speed due to powerful cloud-based infrastructure. Despite this, TextSage provides a balanced performance with acceptable speed and improved accessibility in offline scenarios. Overall, the system proves to be a practical and secure alternative for users prioritizing privacy, cost-efficiency, and offline usability.

Table 7.4: Comparison of TextSage and Online Summarization Tools

Feature	**TextSage**	**Online Tools**
Internet Dependency	Not Required	Required
Data Privacy	High	Low
Cost	Free	Subscription-based
Processing Speed	Moderate	Fast
Accessibility	Offline	Online

"This table presents a comparison between TextSage and online tools based on key factors such as dependency, privacy, cost, speed, and accessibility."

7.5.1 Discussion

TextSage has a great edge in the context of privacy and offline availability. It also guarantees that the data of the users is stored in the local machine, unlike cloud-based solutions, which risks data breaches.Nonetheless, online tools can have higher processing speeds, as they can access powerful cloud infrastructure.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

In the present project, an offline text summarization application was designed and developed successfully with the help of Transformer-based deep learning models. It combines refined versions of PEGASUS, BART and T5 into a graphical user interface which can be run on a desktop computer, allowing users to create summaries without the need to have an internet connection.

A variety of benchmark datasets, including CNN/DailyMail, XSum, and QMSum, were used to optimize the models to ensure good performance on a broad textual domain. The ROUGE-based evaluation showed that the models produce sensible and informative summaries and PEGASUS and BART show high-performing results across the range of datasets. The developed application provides a convenient means of summarizing text locally and making it flexible upon the selection of multi-models and the possibility to work with both manual text input and text files. The project also demonstrates how one can add advanced NLP models to offline applications which are user friendly and how the gap between research and usability can be bridged. In general, the aim of developing, executing and testing an offline summarization system was met successfully.

8.2 Future Scope

Although the proposed system is successful, several ways can be enhanced in the future. The potential area of enhancement is the addition of support to other document formats such as PDF, Word documents and web content. This would increase the practicality of the system in the field. Other possible extensions are the multilingual summarization where the system is capable of summarizing text in more than one language. This would boost the application use and availability. Improvement methods to increase the factual consistency and reduce hallucinations with generated summaries could also be integrated into the system in the future, though this remains an unexplored area of research in abstractive summarization. They could apply performance optimization and model compression techniques such as quantization or distillation to consume fewer resources and execute the system at a lower speed on smaller machines with fewer resources. Finally, it could be extended to a more web-based or cloud-supported hybrid platform to provide collaborative functionality, large-scale processing of documents and real-time summarization of live data.

8.3 Real World Applications

Text summarization systems like TextSage have a wide range of applications across various domains.

8.3.1 Education

- Students and researchers can use the system to summarize lengthy textbooks, research papers, and lecture notes, enabling faster learning and revision.Students can quickly convert lengthy textbooks, research papers, and notes into concise summaries, saving time and improving productivity.Researchers and students can use the system to extract essential information from multiple academic papers, making literature review more efficient.Simplified summaries make complex topics easier to understand, especially for beginners or students from non-technical backgrounds.
- Since the tool works without internet access, it is highly useful in remote areas or low-connectivity environments.
- Students can generate structured notes from lectures, articles, or digital content with minimal manual effort.
- Summarized text can help learners grasp the core meaning of passages, aiding comprehension and vocabulary building.
- Reduces time spent reading large volumes of text, allowing students to focus on understanding and analysis.

8.3.2 Healthcare

- Medical professionals can summarize patient records, clinical reports, and research articles to quickly access important information.
- Doctors can quickly summarize lengthy patient records, clinical reports, and discharge summaries to access key information efficiently.
- Healthcare professionals and researchers can extract essential insights from medical journals and studies, saving time during literature reviews.
- Concise summaries of patient history and reports can assist doctors in making faster and more informed decisions.
- The offline nature of the system makes it highly useful in rural or low-connectivity regions where internet access is limited.
- Reduces the time required to read extensive medical documents, allowing doctors to focus more on patient care.
- Complex medical reports can be summarized into simpler language, helping patients better understand their condition.

- Since all processing is done locally, sensitive medical data remains secure and is not transmitted over the internet.
- Students and trainees can use the tool to summarize case studies and learning materials for quicker understanding.

8.3.3 Legal Sector

- Legal practitioners can use summarization tools to condense lengthy legal documents, case files, and contracts, saving time and improving efficiency.
- Lawyers and legal professionals can quickly summarize lengthy case files, contracts, and legal documents to extract key points efficiently.
- The tool helps in condensing judicial judgments and case laws, making it easier to identify relevant precedents.
- Important clauses, obligations, and risks in contracts can be highlighted through concise summaries.
- Reduces the time required to read and analyze large volumes of legal text, improving overall productivity.
- Ensures that sensitive legal data remains secure, as all processing is performed locally without internet dependency.
- Dense legal jargon can be summarized into more understandable content for clients or non-experts.
- Law students can use the tool to summarize case studies, statutes, and legal materials for better comprehension and revision.

8.3.4 Business and Corporate Sector

- Organizations can summarize meeting transcripts, reports, and emails, improving productivity and decision-making.
- Business professionals can quickly condense lengthy reports, financial statements, and internal documents into concise summaries for faster decision-making.
- The tool can summarize meeting transcripts or notes, highlighting key discussions, decisions, and action items.
- Long email threads can be summarized to extract essential information, improving communication efficiency.
- Analysts can summarize large volumes of market research data, news articles, and competitor reports to identify trends and insights.
- Reduces the time spent reading extensive documents, allowing employees to focus on strategic tasks.
- Sensitive corporate data remains secure as all processing is performed locally without relying on external servers.

8.3.5 Media and Journalism

- Journalists and readers can use summarization tools to quickly extract key information from news articles and reports.
- Journalists can quickly condense lengthy news reports into short, concise summaries for headlines, briefs, or social media posts.
- Editors can summarize multiple sources to create aggregated news content, improving efficiency in content production.
- Helps journalists extract key information from reports, interviews, and archives while preparing stories.
- Enables faster processing of large volumes of incoming information, allowing quicker publication of essential updates.
- Journalists working in remote or low-connectivity areas can still summarize content without internet access.

REFERENCES

[1] A. See, P. J. Liu, and C. D. Manning, “Get to the point: Summarization with pointer-generator net works,” in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2017.

[2] R. Paulus, C. Xiong, and R. Socher, “A deep reinforced model for abstractive summarization,” in *International Conference on Learning Representations (ICLR)*, 2018.

[3] A. Vaswani, N. Shazeer, N. Parmar *et al.*, “Attention is all you need,” in *Advances in Neural Informa tion Processing Systems (NeurIPS)*, 2017.

[4] K. M. Hermann, T. Kocisk ˇ y, E. Grefenstette ´ *et al.*, “Teaching machines to read and comprehend,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

[5] S. Narayan, S. B. Cohen, and M. Lapata, “Don’t give me the details, just the summary! topic-aware convolutional neural networks for extreme summarization,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.

[6] M. Lewis, Y. Liu, N. Goyal *et al.*, “Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.

[7] C. Raffel, N. Shazeer, A. Roberts *et al.*, “Exploring the limits of transfer learning with a unified text to-text transformer,” *Journal of Machine Learning Research*, 2020.

[8] J. Zhang, Y. Zhao, M. Saleh, and P. J. Liu, “Pegasus: Pre-training with extracted gap-sentences for abstractive summarization,” in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.

[9] C. Nnadi and E. Bertini, “A survey on abstractive text summarization: Datasets, models, and metrics,” *arXiv preprint arXiv:2412.17165*, 2024.

[10] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald, “On faithfulness and factuality in abstractive summarization,” in *Proceedings of ACL*, 2020.

[11] M. Cao *et al.*, “Hallucinated but factual! inspecting the factuality of hallucinations in abstractive summarization,” in *Proceedings of ACL*, 2021.

[12] A. Joseph *et al.*, “Factpico: Factuality evaluation for plain language summarization of medical evi dence,” in *Proceedings of ACL*, 2024.

[13] Y. Zhang *et al.*, “A systematic survey of text summarization: From statistical methods to large language models,” *arXiv preprint arXiv:2406.11289*, 2024.

[14] M. Zhong, D. Yin, T. Yu *et al.*, “Qmsum: A new benchmark for query-based multi-domain meeting summarization,” in *Proceedings of NAACL*, 2021.

[15] Y. Mao *et al.*, “Dyle: Dynamic latent extraction for abstractive long-input summarization,” in *Proceed ings of ACL*, 2021.
9[16] Y. Liu and M. Lapata, “Hierarchical transformers for multi-document summarization,” in *Proceedings of ACL*, 2019.
[17] M. Grusky, M. Naaman, and Y. Artzi, “Newsroom: A dataset of 1.3 million summaries with diverse extractive strategies,” in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2018.
[18] A. Chauhan *et al.*, “Topic oriented summarization using transformer-based encoder decoder models,” in *Proceedings of the DialogSum Workshop*, 2022.
[19] A. Jamshed *et al.*, “A novel enhanced digital image correlation framework for structural health moni toring of aircraft,” *Measurement*, p. 119264, 2025.
[20] H. Khatter *et al.*, “An image-based recommendation system using voting-assisted ensemble machine learning approach,” *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, vol. 33, no. 6, pp. 733–759, 2025.
[21] Y. Liu and M. Lapata, “Text summarization with pretrained encoders,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.
[22] Anonymous, “Hegel: Hypergraph transformer for long document summarization,” in *Proceedings of ACL*, 2022.
[23] C.-Y. Lin, “Rouge: A package for automatic evaluation of summaries,” in *Text Summarization Branches Out*, 2004.

Research Paper Acceptance Proof

ICDPN

International Conference on Data-Processing and Networking
(ICDPN-2025)
Web : icdpn-conf.com / Email : icdpn.conf@gmail.com

Date: 07th-08th November 2025

Date: 07/10/2025

Registeration Done by:
Turvash Singh
Department of Computer Science, KIET Group of Institutions, India
turvashsingh@gmail.com

PAPER & PAYMENT DETAILS MADE TO:

The paper title & the amount credited into the account are given below:

Specifications	*Quantity*	*For Publishing & Editing Charges*	*Net Amount (Inclusive of Taxes After Deduction)*
1. Paper Registration details: ICDPN PAPER ID -202513 Paper Title : A Systematic Evaluation of BART, T5, and PEGASUS for Abstractive Summarization Author(s): Turvash Singh, Vinay Singh, Chaitany Saini, Dr. Anurag Mishra	*01*	Rs 12,500/-	Rs 12,500/-
Total Amount			Rs 12,500/-

Payment made online via:
Pay via Bank Transfer:
ACCOUNT NAME : ICICC
ACCOUNT NUMBER: 510909010080523
BANK : City Union Bank
BRANCH : Sector-8, Rohini Delhi
BRANCH CODE : 244
IFSC CODE : CIUB0000244
MICR : 600054027
SWIFT CODE: CIUBIN5M
ACCOUNT TYPE : CURRENT

or Paid via Online (Stripe) :
https://buy.stripe.com/00gcP969V3is1Jm5kn

Organizer,
ICDPN-2025

(ICDPN - 2025)

SPU Slovenská poľnohospodárska univerzita v Nitre | SPU-TF Technická fakulta | National Technical University of Ukraine "Igor Sikorsky Kyiv Polytechnic Institute" | CIT Canadian Institute of Technology | Universal Innovators | Springer

INTERNATIONAL CONFERENCE ON DATA-PROCESSING AND NETWORKING (ICDPN-2025)

Certificate

This is to certify that **Prof. /Dr. / Mr. / Ms. Turvash Singh** is a presenter/co-author of the paper titled **A Systematic Evaluation of BART, T5, and PEGASUS for Abstractive Summarization** presented at the **International Conference on Data-Processing and Networking (ICDPN-2025)** organized jointly by the Institute of Technology and Business (VŠTE) in České Budějovice, Czech Republic, Europe; Slovak University of Agriculture in Nitra, Slovakia; Igor Sikorsky Kyiv Polytechnic Institute, Ukraine; Mata Sundri College for Women, University of Delhi, Delhi & Canadian Institute of Technology, Tirana, Albania, Canada **on 07th – 08th November, 2025**.

Prof. Ing. Jan Valíček, Ph.D.
Publication Chair
Institute of Technology and Business in České Budějovice, Czech Republic , Europe

Prof. Bal Virdee
General Chair
London Metropolitan University, London, United Kingdom

Prof. Abhishek Swaroop
Publication Chair
International Conference on Artificial Intelligence and Networking

(ICDPN - 2025)

SPU Slovenská poľnohospodárska univerzita v Nitre · SPU-TF Technická fakulta · CIT Canadian Institute of Technology · Universal Innovators · Springer

INTERNATIONAL CONFERENCE ON DATA-PROCESSING AND NETWORKING (ICDPN-2025)

Certificate

This is to certify that **Prof. /Dr. / Mr. / Ms. Vinay Singh** is a presenter/co-author of the paper titled **A Systematic Evaluation of BART, T5, and PEGASUS for Abstractive Summarization** presented at the **International Conference on Data-Processing and Networking (ICDPN-2025)** organized jointly by the Institute of Technology and Business (VŠTE) in České Budějovice, Czech Republic, Europe; Slovak University of Agriculture in Nitra, Slovakia; Igor Sikorsky Kyiv Polytechnic Institute, Ukraine; Mata Sundri College for Women, University of Delhi, Delhi & Canadian Institute of Technology, Tirana, Albania, Canada **on 07th – 08th November, 2025**.

Prof. Ing. Jan Valíček, Ph.D.
Publication Chair
Institute of Technology and Business in České Budějovice, Czech Republic , Europe

Prof. Bal Virdee
General Chair
London Metropolitan University, London, United Kingdom

Prof. Abhishek Swaroop
Publication Chair
International Conference on Artificial Intelligence and Networking

(ICDPN - 2025)

SPU Slovenská poľnohospodárska univerzita v Nitre · SPU-TF Technická fakulta · CIT Canadian Institute of Technology · Universal Innovators · Springer

INTERNATIONAL CONFERENCE ON DATA-PROCESSING AND NETWORKING (ICDPN-2025)

Certificate

This is to certify that **Prof. /Dr. / Mr. / Ms. Chaitany Saini** is a presenter/co-author of the paper titled **A Systematic Evaluation of BART, T5, and PEGASUS for Abstractive Summarization** presented at the **International Conference on Data-Processing and Networking (ICDPN-2025)** organized jointly by the Institute of Technology and Business (VŠTE) in České Budějovice, Czech Republic, Europe; Slovak University of Agriculture in Nitra, Slovakia; Igor Sikorsky Kyiv Polytechnic Institute, Ukraine; Mata Sundri College for Women, University of Delhi, Delhi & Canadian Institute of Technology, Tirana, Albania, Canada **on 07th – 08th November, 2025**.

Prof. Ing. Jan Valíček, Ph.D. **Publication Chair** Institute of Technology and Business in České Budějovice, Czech Republic , Europe	**Prof. Bal Virdee** **General Chair** London Metropolitan University, London, United Kingdom	**Prof. Abhishek Swaroop** **Publication Chair** International Conference on Artificial Intelligence and Networking

(ICDPN - 2025)

SPU Slovenská poľnohospodárska univerzita v Nitre · SPU-TF Technická fakulta · VŠTE · National Technical University of Ukraine "Igor Sikorsky Kyiv Polytechnic Institute" · CIT Canadian Institute of Technology · Universal Travelers · Springer

INTERNATIONAL CONFERENCE ON DATA-PROCESSING AND NETWORKING (ICDPN-2025)

Certificate

This is to certify that **Prof. /Dr. / Mr. / Ms. Anurag Mishra** is a presenter/co-author of the paper titled **A Systematic Evaluation of BART, T5, and PEGASUS for Abstractive Summarization** presented at the **International Conference on Data-Processing and Networking (ICDPN-2025)** organized jointly by the Institute of Technology and Business (VŠTE) in České Budějovice, Czech Republic, Europe; Slovak University of Agriculture in Nitra, Slovakia; Igor Sikorsky Kyiv Polytechnic Institute, Ukraine; Mata Sundri College for Women, University of Delhi, Delhi & Canadian Institute of Technology, Tirana, Albania, Canada **on 07th – 08th November, 2025**.

Prof. Ing. Jan Valíček, Ph.D. **Publication Chair** Institute of Technology and Business in České Budějovice, Czech Republic , Europe	**Prof. Bal Virdee** **General Chair** London Metropolitan University, London, United Kingdom	**Prof. Abhishek Swaroop** **Publication Chair** International Conference on Artificial Intelligence and Networking

Research Paper

Project Report Plagiarism Report

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.